

Contents

Open-end shock-tube change-detection analysis	1
Table of contents	1
Quick start	2
How it works (the core idea)	2
Installation / requirements	3
Camera metadata (.cihx) auto-detection	4
Script 1 — detect_activity.py	4
Script 2 — analyze_activity.py	6
Shared module — change_common.py	8
Tuning guide	9
Troubleshooting	9
Validation	9

Open-end shock-tube change-detection analysis

Detect and characterise any change in high-speed-camera recordings of the open end of a shock tube — the moments when flow comes out of the tube and something happens outside it: a plume, a density gradient, ejected particles, a faint shimmer.

A recording is a directory of thousands-to-millions of single-frame TIFFs from a Photron FASTCAM. Each frame is a 2-D grayscale image. For most of the recording the scene is quiescent — a static background of the tube exit and its surroundings. At some point the flow exits and the image changes. Unlike the sister project [shock_tube_image_analysis](#) — which looks for a specific signature (a thin moving shock line) — this pipeline assumes no shape. It subtracts the quiescent background and flags any spatially-coherent departure from it, however faint.

The pipeline has two stages:

1. **detect_activity.py** — streams through every frame and finds the stretches where the scene changes (the “events”).
2. **analyze_activity.py** — reloads each detected event and measures what the change is doing: strength, location, footprint, centroid motion and speed, growth/decay, polarity (brighter or darker than the background).

A shared module, **change_common.py**, holds the image I/O, the background/noise modelling, the change-detection signal processing, and the Photron .cihx metadata parsing used by both. (The .cihx parsing is shared verbatim with the sister project, so a recording’s camera metadata is read the same way by both pipelines.)

Table of contents

- [Quick start](#)
- [How it works \(the core idea\)](#)
- [Installation / requirements](#)
- [Camera metadata \(.cihx\) auto-detection](#)
- [Script 1 — detect_activity.py](#)
- [Script 2 — analyze_activity.py](#)
- [Shared module — change_common.py](#)
- [Tuning guide](#)
- [Troubleshooting](#)
- [Validation](#)

Quick start

```
# 1. Detect events. fps is auto-read from the .cihx sidecar in the folder.
# Add --plot-timeline to see the whole-movie activity curve.
python3 detect_activity.py RUN --out activity_events.csv --plot-timeline timeline.png

# 2. Characterise each detected event, with diagnostic plots.
python3 analyze_activity.py RUN activity_events.csv --out-dir analysis --plots

# ...or also export a folder of treated per-frame images and an MP4 per event.
python3 analyze_activity.py RUN activity_events.csv --out-dir analysis \
    --plots --dump-frames --movie
```

To get speeds in m/s (and reach in mm), add a pixel size if the recording is not spatially calibrated in PFV (the common case):

```
python3 analyze_activity.py RUN activity_events.csv --out-dir analysis --plots --px-size-um 50
```

That's it. `detect_activity.py` writes `activity_events.csv`; `analyze_activity.py` writes a per-event summary, per-frame measurements, and diagnostic PNGs into the output directory. If the recording is quiescent throughout, `activity_events.csv` contains only its header — the correct result for “nothing happened”.

How it works (the core idea)

The whole method rests on one distinction. A pixel at frame t can differ from the static scene for two reasons:

1. A genuine, spatially-coherent change — the flow. Many neighbouring pixels move together, even if each only slightly.
2. Independent per-pixel sensor noise — each pixel wanders on its own.

A single pixel of noise is indistinguishable from a single pixel of faint flow. What separates them is spatial coherence — the spatial analogue of the temporal coherence the sister project uses to recognise a shock. We exploit it in four steps:

```
raw frame (H x W)
  | subtract per-pixel background          → removes the static scene
  ▼
difference image
  | spatial band-pass (narrow – wide box) → removes illumination
  |                                         flicker / optical drift
  ▼
detrended difference
  | divide by per-pixel noise sigma,      → ~N(0,1) where quiescent
  | pooled over a small box (smooth)      coherent change survives
  ▼
pooled SNR map → count pixels with |SNR| > threshold = "activity"
  | per frame, streamed
  ▼
group frames where activity persists → activity_events.csv
```

Two nuisance signals are removed along the way, exactly as in the sister project but in 2-D:

- The fixed spatial pattern (tube walls, optics, illumination) → removed by subtracting a per-pixel background, estimated robustly (median) from the quiescent frames.

- A slow illumination drift / global flicker → removed by the spatial band-pass (subtract a wide local-mean of the difference image). A global brightness wobble is coherent across the whole frame and would otherwise swamp the pooled SNR; a localised plume is small compared to the wide box and survives.

The hard part — telling one frame of faint flow from one frame of noise — is answered by demanding coherence on two axes at once:

- Spatial: an event frame must contain a contiguous region of enough active pixels (`--min-area`), not stray noise specks.
- Temporal: the change must persist over several consecutive frames (`--min-len`), not flicker for a single frame.

This is what keeps a quiescent recording from producing false positives.

Robust to flow of unknown duration

The background and the per-pixel noise are estimated from a uniform temporal sample of frames (so memory is bounded regardless of recording length), then refined: the sample frames that turn out to be active are dropped and the model is rebuilt from the rest. So even if the flow lasts a large fraction of the recording, as long as enough genuinely-quiescent frames exist somewhere, they define the background — the flow is never absorbed into it. You do not need to know in advance how long the ejection lasts.

Self-calibrating threshold

Real camera noise is not perfectly independent (row noise, residual fixed pattern), so the pooled SNR is not exactly $N(0,1)$. The model measures the actual pooled-SNR spread on the quiescent frames (the inflation factor, ideally ~ 1) and scales the pixel threshold by it, so the per-pixel false-positive rate is well-defined whatever the camera's quirks.

Installation / requirements

- Python 3.9+
- NumPy
- Pillow (PIL) — TIFF reading
- Matplotlib — only for `--plots` / `--plot-timeline` / `--dump-frames` / `--movie`
- ffmpeg — only for `--movie` (MP4 export); must be on PATH. Not a pip package: `brew install ffmpeg` (macOS) or `sudo apt install ffmpeg` (Debian/Ubuntu).

No SciPy / OpenCV / tiffle required. Install the Python deps with:

```
python3 -m pip install -r requirements.txt
```

The detector reads frames one at a time during its streaming pass and keeps only the fixed-size background maps plus a bounded temporal sample in memory, so it scales to very long recordings. Memory is $O(\text{--bg-sample})$ frames, not $O(\text{total frames})$.

Data layout

```
RUN/
├─ RUN.cihx          # Photron camera-metadata sidecar (optional but recommended)
├─ RUN000001.tif     # frames, named so alphabetical order == temporal order
├─ RUN000002.tif
└─ ...
```

Frame files must sort alphabetically into temporal order (zero-padded numbers do this automatically). The integer frame number is taken from the last run of digits in each filename. Frames may be any 2-D size; RGB frames are collapsed to luminance.

Camera metadata (**.cihx**) auto-detection

If a Photron **.cihx** (or **.cih**) file is present in the recording directory, both scripts read it automatically — you do not need to pass camera parameters on the command line.

Field read from .cihx	Used for
recordRate (fps)	event duration in μ s, speed in px/s and m/s
resolution	reported in the log
colorInfo/bit (depth)	reported in the log
shutterSpeedNsec	exposure time, reported
deviceName	reported in the log
plugin/calibration	spatial pixel size (m/s, mm output)

Frame rate (**fps**) is always applied — it is reliable. Spatial pixel size is only applied when the recording is genuinely calibrated. The PFV default (`scaleMode = 0`, `sizeOfPixel = 1.0`, `magnification = 1.0`) is treated as uncalibrated: the scripts deliberately refuse to invent a physical scale from that placeholder, log a note, and report motion in px/frame and px/s only. To get m/s either calibrate the scale in PFV, or pass `--px-size-um`.

Command-line `--fps` / `--px-size-um` always override the **.cihx**.

Script 1 — **detect_activity.py**

Purpose: scan a whole recording and output one row per stretch where the scene changes.

Usage

```
python3 detect_activity.py INPUT_DIR [options]
# Simplest: fps auto-read from the .cihx
python3 detect_activity.py RUN --out activity_events.csv

# Restrict to a region of interest and dump the activity timeline for inspection
python3 detect_activity.py RUN --row-lo 40 --row-hi 360 \
    --save-activity activity.npz --plot-timeline timeline.png

# More sensitive (catch fainter flow), with a larger pooling window
python3 detect_activity.py RUN --pix-k 5 --smooth 7 --min-area 12
```

What it does

1. Lists frames in temporal order.
2. Builds the quiescent-scene model — a per-pixel background and per-pixel noise — from a uniform sample of `--bg-sample` frames, refined to exclude any active frames (`change_common.build_background`).

3. Streams through every frame, forms the band-passed, noise-normalised pooled SNR map, and reduces it to a per-frame activity score: the number of pixels whose pooled SNR exceeds the (inflation-scaled) `--pix-k`.
4. Flags frames whose active-pixel area reaches `--min-area`, and groups consecutive flagged frames into events — bridging gaps up to `--max-gap`, keeping those of length within `[--min-len, --max-len]`.

Options

Option	Default	Meaning
<code>input_dir</code>	—	Directory of single-frame TIFFs (positional).
<code>--pattern</code>	<code>*.tif</code>	Glob selecting frame files.
<code>--out</code>	<code>activity_events.csv</code>	Output CSV path.
Region of interest		
<code>--row-lo / --row-hi</code>	<code>all</code>	Restrict the analysed rows.
<code>--col-lo / --col-hi</code>	<code>all</code>	Restrict the analysed columns.
<code>--edge-margin</code>	<code>6</code>	Pixels ignored at each border (unreliable optics edge).
Background / noise model		
<code>--bg-sample</code>	<code>150</code>	Frames sampled uniformly to build the background.
<code>--noise-floor-frac</code>	<code>0.25</code>	Floor on per-pixel σ (fraction of the median σ).
<code>--no-refine-bg</code>	<code>off</code>	Skip dropping active frames from the background.
<code>--refine-k</code>	<code>4.0</code>	Robustness of the active-frame cut during refinement.
Change detection		
<code>--smooth</code>	<code>5</code>	Side of the spatial box pooling the SNR map. Larger = more sensitive to faint, broad change.
<code>--detrend-band</code>	<code>101</code>	Wide box subtracted from the difference image (band-pass) to remove flicker/drift. Make it $\gg$ the change size; <code>0</code> disables.
<code>--pix-k</code>	<code>6.0</code>	Per-pixel pooled-SNR threshold (σ). Lower (≈ 5) for more sensitivity; raise to reject noise.
<code>--min-area</code>	<code>auto</code>	Active-pixel count for a frame to be “active”. Auto = $\max(8, \text{frac} \cdot H \cdot W)$.
<code>--min-area-frac</code>	<code>5e-5</code>	Sets the auto <code>--min-area</code> .
Event grouping		
<code>--min-len</code>	<code>3</code>	Min consecutive active frames per event (temporal persistence).
<code>--max-len</code>	<code>none</code>	Max frame span of an event (unlimited by default — flow can last).
<code>--max-gap</code>	<code>2</code>	Max inactive frames bridged within an event.

Metadata / diagnostics

Option	Default	Meaning
<code>--fps</code>	from <code>.cihx</code>	Camera frame rate; overrides the sidecar.
<code>--save-activity</code>	off	Dump the per-frame activity arrays to an <code>.npz</code> .
<code>--plot-timeline</code>	off	PNG of activity-vs-frame for the whole movie.

Output: **activity_events.csv**

One row per detected event:

Column	Description
<code>event_id</code>	0-based index, referenced by Script 2.
<code>start_frame</code> , <code>end_frame</code> , <code>peak_frame</code>	Frame numbers of the first / last / most-active frame.
<code>n_active</code>	Number of active frames in the event.
<code>span</code>	<code>end_index - start_index + 1</code> (including bridged gaps).
<code>peak_area_px</code> , <code>mean_area_px</code>	Active-pixel area at the peak / on average.
<code>peak_snr</code>	Strongest pooled SNR in the event.
<code>peak_energy</code>	Largest integrated
<code>polarity</code>	brighter or darker than the background.
<code>onset_cx</code> , <code>onset_cy</code> , <code>peak_cx</code> , <code>peak_cy</code>	Change centroid (px) at onset / peak.
<code>start_index</code> , <code>end_index</code> , <code>peak_index</code>	Internal 0-based frame ordinals (for debugging).

If no events are found the file contains just the header row.

Script 2 — **analyze_activity.py**

Purpose: take the events found by Script 1 and measure each change precisely.

Usage

```
python3 analyze_activity.py INPUT_DIR EVENTS_CSV [options]
python3 analyze_activity.py RUN activity_events.csv --out-dir analysis \
    --px-size-um 50 --plots
```

What it does

For each event it reloads only that event's frames plus a **--margin** of context frames, then:

1. Builds a local background + noise from the margin frames (excluding the core event frames so the change does not bias the baseline), with its own inflation calibration.
2. Measures, per frame: active-pixel area, largest connected blob, change centroid (x , y), bounding box, leading-edge distance from the onset centroid, pooled peak/integrated SNR, and polarity.
3. Summarises the event: onset/peak/end, duration, peak and mean area, peak SNR, maximum reach, and a descriptive centroid-trajectory fit giving the velocity vector (v_x , v_y) and speed. (There is no straight-line acceptance gate — the fit is a summary, not a filter.)

4. Converts to physical units when available: $\text{speed}[\text{m/s}] = \text{speed}[\text{px/frame}] \times \text{fps} \times \text{pixel_size}[\text{m}]$, $\text{duration}[\mu\text{s}] = \text{duration}[\text{frames}] / \text{fps}$, reach in mm.
5. Optionally renders two diagnostic PNGs per event (`--plots`), a folder of treated per-frame images (`--dump-frames`), and/or an MP4 movie (`--movie`).

Keep the detection options (`--smooth`, `--detrend-band`, `--pix-k`, `--edge-margin`, `--row/col-lo/hi`) consistent with what you used in Script 1 so the measurements match.

Options

Option	Default	Meaning
<code>input_dir</code>	—	Directory of single-frame TIFFs (positional).
<code>events_csv</code>	—	CSV produced by Script 1 (positional).
<code>--pattern</code>	<code>*.tif</code>	Glob selecting frame files.
<code>--out-dir</code>	<code>analysis</code>	Output directory (created if needed).
<code>--row-lo/hi, --col-lo/hi</code>	<code>all</code>	Region of interest (match Script 1).
<code>--edge-margin</code>	<code>6</code>	Border pixels ignored (match Script 1).
<code>--smooth</code>	<code>5</code>	Spatial pooling window (match Script 1).
<code>--detrend-band</code>	<code>101</code>	Band-pass width (match Script 1).
<code>--pix-k</code>	<code>6.0</code>	Per-pixel SNR threshold (match Script 1).
<code>--noise-floor-frac</code>	<code>0.25</code>	Floor on per-pixel σ .
<code>--margin</code>	<code>10</code>	Context frames padded around each event.
<code>--fps</code>	from <code>.cihx</code>	Camera frame rate; overrides the sidecar.
<code>--px-size-um</code>	from <code>.cihx</code> if calibrated	Pixel size in μm , for m/s and mm output.
<code>--plots</code>	<code>off</code>	Write the montage + trajectory diagnostic PNGs per event.
<code>--dump-frames</code>	<code>off</code>	Write one treated image per frame (raw frame + detected-region outline + centroid) into a per-event subfolder, over the full <code>core+margin</code> window.
<code>--movie</code>	<code>off</code>	Render an MP4 per event from the same treated frames (needs <code>ffmpeg</code>).
<code>--movie-fps</code>	<code>12</code>	Playback frame rate of the MP4. (Capture is much faster, so this slows the action down.)

Outputs (in `--out-dir`)

events_summary.csv — one row per event:

Column	Description
event_id	Matches activity_events.csv.
start_frame, peak_frame, end_frame	First / most-active / last frame.
n_frames_active, duration_frames	Active-frame count and total span.
polarity	brighter / darker.
peak_area_px, mean_area_px	Footprint at the peak / on average.
peak_snr	Strongest pooled SNR.
max_leading_edge_px	Furthest reach of the active region from the onset centroid.
onset_cx/cy, peak_cx/cy	Centroid at onset / peak.
centroid_travel_px	Straight-line centroid displacement.
centroid_vx/vy_px_per_frame, centroid_speed_px_per_frame	Centroid velocity vector and speed.
centroid_speed_px_per_s, duration_us	If fps known.
centroid_speed_m_per_s, max_leading_edge_mm	If fps and pixel size known.

event_<id>_frames.csv — per-frame detail across the analysis window: frame, order_index, in_core, area_px, blob_px, centroid_x/y, bbox_w/h, leading_edge_px, peak_snr, energy, mean_snr, polarity. Margin frames have in_core = False.

event_<id>_montage.png — (with --plots) a strip of pooled-SNR difference images across the event (red = brighter, blue = darker) with the active region outlined in green and the centroid marked.

event_<id>_trajectory.png — (with --plots) the centroid trajectory (coloured by frame) and the active-area-vs-frame growth/decay curve.

event_<id>_frames/ — (with --dump-frames) a folder of one PNG per frame across the full analysis window (detected core plus the $\pm$ --margin context frames): the raw grayscale camera frame with the detected active region outlined in green and the centroid marked with a red X, titled with the frame number, a core/margin tag, the active area and the polarity. Files are zero-padded (frame_004001206.png) so they sort into temporal order and play as a flip-book.

event_<id>.mp4 — (with --movie) the same treated frames rendered as an H.264 movie, one video frame per camera frame, at --movie-fps (default 12).

Shared module — **change_common.py**

Imported by both scripts; not run directly. Key contents:

Frame discovery & I/O - list_frames(input_dir, pattern) — frame paths in temporal order. - frame_number(path) — integer frame number from a filename. - load_frame(path, row_lo, row_hi, col_lo, col_hi) — load a frame as a 2-D float image, optionally cropped.

Background / noise & change detection (pure NumPy, no SciPy) - box2d_mean(img, k) — k×k box average via a summed-area table, O(H·W). - estimate_bg_noise(stack) — robust per-pixel median background and MAD noise. - build_background(files, p) → BackgroundModel — sampled, refined background, noise, and the inflation-scaled pixel threshold. - activity_snr(frame, bg, sigma2, smooth, de-trend_band) — band-passed, noise-normalised pooled SNR map. - measure_activity(snr, thresh, edge_margin) → ActivityMeasurement. - largest_blob(mask) — largest connected component (flood fill). - robust_threshold(scores, k) — median + k·(1.4826·MAD).

Photron metadata (shared verbatim with the sister project) - `find_cihx / parse_cihx → CameraMeta. - resolve_calibration(input_dir, fps_arg, px_size_um_arg).`

Tuning guide

Start from the defaults; they are set so a quiescent recording yields zero events. When you have a recording that does contain flow and detection is off, adjust in this order:

Symptom	Knob	Direction
Faint flow missed entirely	<code>--pix-k</code>	lower (e.g. 6 → 5) — more sensitive pixel threshold
Faint, broad flow missed	<code>--smooth</code>	raise (e.g. 5 → 7–9) — more spatial pooling
Flow detected but split into pieces	<code>--max-gap</code>	raise (e.g. 2 → 4) to bridge dropouts
Short events rejected	<code>--min-len</code>	lower (e.g. 3 → 2)
Too many false positives (noise)	<code>--pix-k, --min-len, --min-area</code>	raise — stricter coherence
Illumination flicker leaking through	<code>--detrend-band</code>	keep it >> the change size; lower if drift is finer-scale
A diffuse, frame-filling glow is suppressed	<code>--detrend-band</code>	raise (or 0) — it is being band-passed away
Only the tube exit matters	<code>--row/col-lo/hi</code>	restrict the ROI to cut spurious triggers elsewhere

Use `--save-activity activity.npz` and `--plot-timeline timeline.png` to inspect the activity signal and choose `--pix-k` / `--min-area` deliberately. The detector logs the inflation factor and the effective pix threshold — if inflation is far above 1, the per-pixel noise is strongly correlated and a larger `--smooth` or `--detrend-band` will help.

Troubleshooting

- “no .cihx/.cih sidecar found” — fine; just pass `--fps` (and optionally `--px-size-um`) yourself.
 - “recording is NOT spatially calibrated” — expected when the scale wasn’t set in PFV. Motion is still reported in px/frame and px/s; add `--px-size-um` for m/s and mm.
 - Whole recording flagged as one giant event — the background absorbed something it shouldn’t, or strong global flicker is leaking through. Check the logged inflation factor; raise `--detrend-band`, and make sure enough quiescent frames exist for `--bg-sample`.
 - Zero events on a recording you believe has flow — see the tuning table; most often lower `--pix-k` and/or raise `--smooth`. Confirm the flow really is a departure from the background and not within the per-pixel noise.
 - **FileNotFoundError: No frames matching ...** — check `--pattern` and the directory path.
 - Speeds look wrong by a constant factor — check `--px-size-um` and that `--fps` matches the recording (not a default).
-

Validation

The pipeline ships validated against two cases built from the sister project’s real T1 background (`_make_test_data.py` regenerates them):

- True negative: 160 untouched, quiescent T1 frames produce 0 events at the defaults — the background subtraction, band-pass and two-axis coherence gate reject the residual camera noise. (Lowering `--pix-k` to 5 surfaces a single marginal 3-frame noise-tail excursion, which `--pix-k 6` or `--min-len 4` removes — a useful illustration of the sensitivity/robustness trade-off.)
- True positive: a faint moving Gaussian “plume” (peak ≈ 60 DN over ~ 10 DN noise) injected into the real background, brightening then fading over 14 frames while moving at 45 px/frame, is recovered with the correct extent, polarity (brighter), and kinematics — centroid velocity 45.02 px/frame (≈ 388.9 m/s at 172 800 fps and $50\text{ }\mu\text{m/px}$), $v_y \approx 0$.